

Welcome back! Aapne Module 1 aur 2 bohot achhe se cover kar liye. Ab RAG pipeline ko cost-effective aur hyper-focused banane ka time aagaya hai.

Aaj hum deep dive karenge **Module 3: Contextual Compression** mein.

Module 3: Contextual Compression

1. Why this concept exists

Jab hum vector search se documents nikalte hain, toh wo fix-sized chunks hote hain (e.g., 1000 tokens). Par kya user ke answer ko un poore 1000 tokens ki zaroorat hoti hai? Nahi. Kabhi-kabhi answer sirf ek sentence (10 tokens) mein chhupa hota hai. Contextual Compression exist karta hai taaki hum us 1000-token ke chunk mein se sirf wo 10-token ka "juice" (relevant part) nikal sakein aur baaki ka "kachra" (noise) hata sakein.

2. Problem with traditional RAG

Basic RAG mein ek bohot badi problem hai: **Noise aur Cost**.

Maan lo aapne top 5 chunks retrieve kiye, aur har chunk 1000 tokens ka hai. Aapne LLM ko 5000 tokens bhej diye.

Isse do nuksaan hote hain:

1. **Lost in the middle problem:** LLM itne bade text mein confuse ho jata hai aur important details miss kar deta hai.
2. **High API Cost:** Har baar 5000 tokens process karne ki cost bohot zyada aati hai, jabki kaam ka context sirf 200 tokens ka tha.

3. Real-world example

Aap ek Financial Report (100 pages) padh rahe ho.

User Query: "Q3 mein Company ka profit kitna tha?"

Traditional RAG: Vector DB ek bada chunk le aayega jisme Q1, Q2, Q3 aur Q4 sabka data hoga, plus CEO ka lamba sa bhashan bhi hoga.

Contextual Compression RAG: Yeh system us bade chunk ko LLM se filter karwayega.

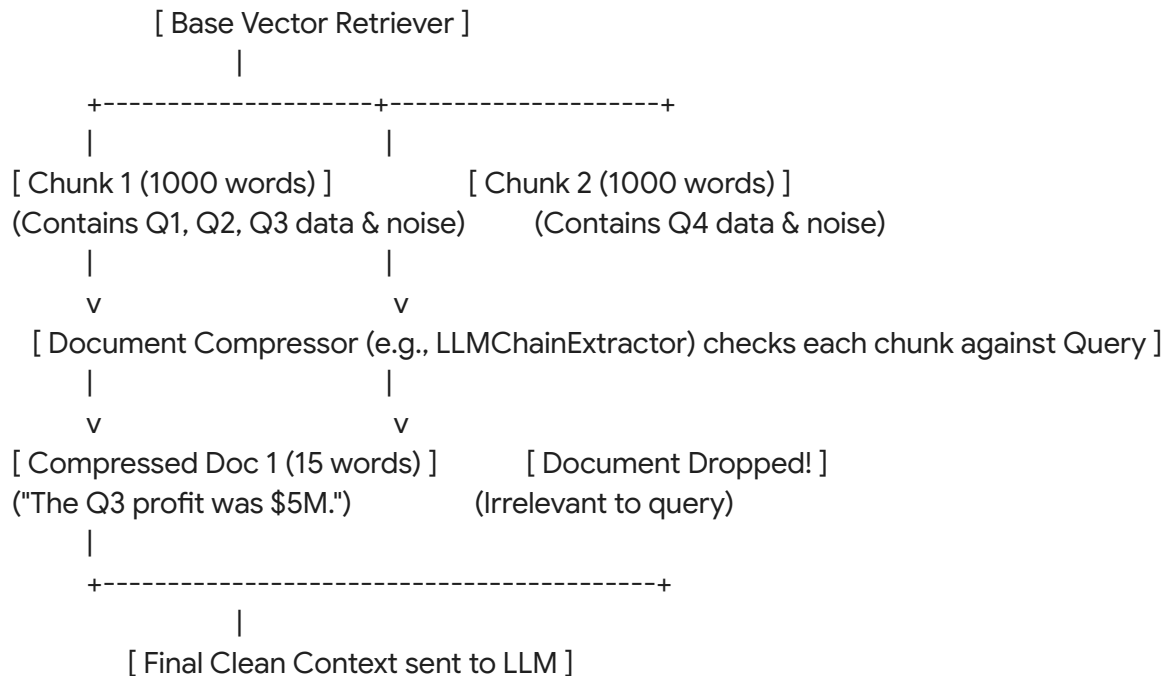
Compressor chunk ko padhega aur sirf Q3 ke profit wali line ko extract karke final LLM ko dega. Boom! Noise gone.

4. Visual explanation using diagrams made with text

[User Query: "What is the Q3 profit?"]

|

v



5. Architecture flow

1. User question puchta hai.
2. Base retriever (FAISS/Chroma) hamesha ki tarah top-K (say, 10) documents retrieve karta hai.
3. Wo 10 documents seedha main LLM ke paas nahi jaate. Wo pehle **Document Compressor** ke paas jaate hain.
4. Compressor (jo khud ek chhota LLM ya filter ho sakta hai) har document ko query ke context mein padhta hai.
5. Jo document bilkul useless hota hai, wo **drop** ho jata hai.
6. Jo document partially useful hota hai, usko **compress/extract** kar liya jata hai (sirf kaam ki line bachti hai).
7. Final compressed, pure context main LLM ko bheja jata hai final answer bananane ke liye.

6. Step-by-step implementation

1. Vector Database (Base Retriever) setup karein.
2. Ek base LLM banayein jo extraction (compression) ka kaam karega.
3. `LLMChainExtractor.from_llm(llm)` initialize karein.
4. `ContextualCompressionRetriever` class use karein, isme `base_compressor` aur `base_retriever` pass karein.
5. Apni query ko is compression retriever ke through run karein.

7. Complete Python code & LangChain implementation

```

import os
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma

# Contextual Compression imports
from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain.retrievers import ContextualCompressionRetriever
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# 1. Setup API Keys
os.environ["OPENAI_API_KEY"] = "sk-your-openai-api-key"

# 2. Let's create a dummy document file first (for testing)
dummy_text = """
The company was founded in 2010 by John Doe. It started as a small shoe store.
Over the years, it expanded into apparel and electronics.
In 2023, the CEO announced a new sustainability initiative.
The Q3 profit in 2023 was a record-breaking $5.2 Million, driven mainly by electronics sales.
Our main headquarters is located in New York, but we have branches in London and Tokyo.
"""

with open("dummy_data.txt", "w") as f:
    f.write(dummy_text)

# Load the document
loader = TextLoader("dummy_data.txt")
docs = loader.load()

# 3. Create Chunks and Vector Store
# Hum intentionally bade chunks bana rahe hain taaki noise aaye
text_splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=0)
splits = text_splitter.split_documents(docs)

embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)

```

```

# 4. Setup Base Retriever
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 5. Setup the Compressor (The Magic Happens Here)
# Hum GPT-3.5 ko bol rahe hain: "Chunks padho aur sirf query se related lines extract karo"
llm_compressor = ChatOpenAI(temperature=0, model="gpt-3.5-turbo")
compressor = LLMChainExtractor.from_llm(llm_compressor)

# 6. Combine Base Retriever and Compressor
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever
)

# 7. Test out the Compression!
query = "What was the Q3 profit in 2023?"

print(f"User Query: {query}\n")

# A. Without Compression (Pura chunk aayega)
print("--- NO COMPRESSION (Base Retriever) ---")
base_docs = base_retriever.invoke(query)
for doc in base_docs:
    print(f"Content: {doc.page_content}")
    print("-" * 20)

# B. With Compression (Sirf relevant part aayega)
print("\n--- WITH COMPRESSION (Filtered Docs) ---")
compressed_docs = compression_retriever.invoke(query)
for doc in compressed_docs:
    print(f"Content: {doc.page_content}")
    print("-" * 20)

# (Optional) Final LLM Chain jaise humesha banate hain, wese hi banegi
# Just use `compression_retriever` instead of `base_retriever` in create_retrieval_chain

```

8. Production use case

Legal Contract Analysis: Lawyers badi contracts search karte hain ("Penalty for early termination"). Contract ke chunk mein 'termination' ke saath 'insurance', 'force majeure', wagera ka useless text bhi aata hai. LLMChainExtractor automatically baki baatein uda deta hai aur sirf 2 line ka penalty clause return karta hai. LLM hallucinate kam karega aur answer to-the-point aayega.

9. Interview questions

1. **Q: LLMChainExtractor ka performance (latency) par kya asar padta hai?**
A: Latency bahut badh jaati hai. Agar base retriever 10 documents lata hai, toh compressor 10 times LLM ko call karta hai sequentially (ya parallel) unhe compress karne ke liye. Yeh slow ho sakta hai.
2. **Q: Agar hume latency bachaani ho, toh hum LLMChainExtractor ki jagah kya use kar sakte hain?**
A: Hum EmbeddingsFilter ya DocumentCompressorPipeline use kar sakte hain, jo purely vector similarity ke basis par relevance check karte hain bina text generate kiye, jo ki bohot fast hota hai.
3. **Q: Contextual Compression aur Parent Document Retriever mein kya farq hai?**
A: Parent Document chunk ko *badha* karta hai (to give more context), jabki Contextual Compression chunk ko *chhota* karta hai (to remove noise).

10. Advantages

- **Token Optimization:** Main LLM ke paas jaane wale tokens drastic amount mein kam ho jate hain (Cost bachati hai).
- **Focus:** "Lost in the middle" problem khatam ho jati hai.
- **Clean Data:** LLM ko sirf quality data milta hai, hallucinations reduce hote hain.

11. Limitations

- **Latency Delay:** LLM-based compressors (like LLMChainExtractor) time lagate hain kyunki wo chunk ko summarize/extract karte hain.
- **Initial API Calls:** Document compress karne ke liye bhi LLM/API calls lagti hain.
- **False Negatives:** Kabhi-kabhi compressor galati se useful information ko kachra samajh kar hata deta hai.

12. Best practices

- **Don't use Heavy LLMs for Compression:** Kabhi bhi GPT-4 ya Claude-3-Opus ko compressor matt banao. Yeh bohot mehnga aur slow padega. Use fast models like Llama-3-8B, GPT-3.5-Turbo, or Claude-Haiku.
- **Filter before Extract:** LangChain mein ek concept hai EmbeddingsFilter. Pehle chunks ko embedding ke basis par strictly filter out karo, uske baad bache hue 2-3 chunks ko hi LLMChainExtractor mein dalo. Isse pipeline fast rehti hai.

Assignments & Practical Exercises

- **Exercise 1:** Upar diye code ko run karo aur dono outputs (NO COMPRESSION vs WITH COMPRESSION) ka difference dhyan se observe karo.
- **Exercise 2:** Research on LangChain Docs: EmbeddingsFilter kya hota hai? Usko ContextualCompressionRetriever mein implement karne ka code try karo (Hint: Isme OpenAIEmbeddings() pass hota hai as a compressor instead of an LLM).

Common Mistakes Beginners Make

- *Applying compression on hundreds of retrieved documents:* Agar base retriever ko k=50 bola, aur compressor lagaya, toh 50 API calls jayengi compress hone ke liye. System crash ho jayega. Always keep base k around 10-15 max.
- *Using the same heavy LLM for everything:* Answer generation wala LLM aur compressor LLM same hona zaroori nahi hai. Engineer as a system!

Yeh tha hamara Module 3. Ab tak ka safar kaisa laga?